

Capstone Project – Step 4 Plan Database Model

1. Introduction

This document presents the database schema design and data model for BindFlow using MongoDB with Mongoose ODM. The design is based on the real-world manual processes of the family business (Orders book, Quotes book, and Maquila/Production book). Payments are kept deliberately simple and manual — staff will verify payments themselves and simply click “Mark Payment Received” in the dashboard. No automation or bank integration is included.

2. Database Choice

Database: MongoDB (NoSQL document database)

ODM: Mongoose (for schema validation, relationships, and easy querying)

Why MongoDB? Flexible schema for varying order specifications and excellent support for embedded arrays (deliveries).

3. Database Models (Mongoose Schemas)

3.1 User Model

```
js
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // hashed with bcrypt
  role: {
    type: String,
    enum: ['client', 'staff', 'admin'],
    default: 'client'
  },
  phone: { type: String },
  address: { type: String },
  businessName: { type: String },
}, { timestamps: true });

const User = mongoose.model('User', userSchema);
```

3.2 Order Model (Central Hub)

```
const orderSchema = new mongoose.Schema({
  client: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },

  // Book information
  book: {
    title: { type: String, required: true },
    author: String,
    isbn: String,
    coverImage: String,
    googleBooksId: String
  },

  // Binding specifications
  specifications: {
    quantity: { type: Number, required: true },
    sheetsPerBook: { type: Number, required: true },
    spiralSideLength: { type: Number, required: true },
    coverMaterial: { type: String, required: true },
    coverThickness: Number,
    bindingType: { type: String, enum: ['spiral', 'perfect', 'saddle-stitched', 'other'] },
    additionalNotes: String
  },

  status: {
    type: String,
    enum: ['pending', 'quoted', 'in-production', 'ready', 'partially-delivered', 'delivered', 'cancelled'],
    default: 'pending'
  },

  // Quote (simple embedded object)
  quote: {
    totalPrice: Number,
    calculatedAt: Date,
    breakdown: mongoose.Schema.Types.Mixed
  },
});
```

```

// Manual Payment (simplified as requested)
payment: {
  status: {
    type: String,
    enum: ['pending', 'received', 'verified'],
    default: 'pending'
  },
  amount: { type: Number },
  receivedDate: { type: Date },
  method: { type: String }, // e.g. "bank-transfer", "cash"
  verifiedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  notes: { type: String }
},

// Production / Maquila tracking
productionNotes: String,
supervisor: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },

}, { timestamps: true });

const Order = mongoose.model('Order', orderSchema);

```

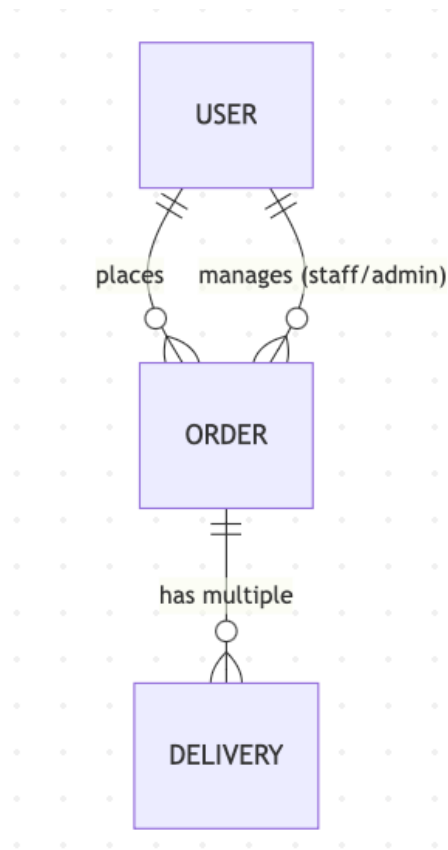
3.3 Delivery Model (Embedded Sub-Document)

```

js
const deliverySchema = new mongoose.Schema({
  deliveryDate: { type: Date, required: true },
  quantityDelivered: { type: Number, required: true },
  signedBy: String,
  signatureImage: String, // Cloudinary URL (stretch)
  notes: String
});

```

4. Relationships (ER Diagram Style)



5. Data Flow & Indexing Strategy

- Indexes: client, status, and payment.status fields in Order
- Book data from Google Books API is saved directly into the Order document.

6. Potential Challenges & Shortcomings

Challenge	Proposed Solution
Manual payment verification	Simple “Mark Payment Received” button in staff dashboard
Complex quoting logic	Pricing algorithm in Mongoose pre-save middleware
Partial deliveries	Embedded array inside Order
Role-based access control	Backend middleware + frontend protection